Developing a virtual assistant for streamlining the learning process of AI image generation in *Automatic1111*.

Emanuela Agnoni - June 2024

Introduction

The “Guiding Star” is a Python based system that uses Retrieval Augmented Generation (RAG) to extract relevant information from a database of knowledge, delivering it back to the user upon request. Its name derives from one of the most ancient systems of navigation and it is inspired by Media.Monks design (Fig.1). The system works in the context of Stable Diffusion (SD), an open source deep learning text-to-image model for AI image generation (AIG), which runs locally on the user’s computer. The user can operate SD via its interface, called Automatic1111(A11), which is open source and was chosen among others as it is the most commonly used and preferred choice among the users interviewed. The data that the system uses to inform its replies comes from the official repository of A11 on Github, and more conceptual data was later added, such as tutorials and guides.

A11 is quite a technical interface which lacks guidance, making it often overwhelming for beginner users. AIG is also inherently complex: between crafting the right prompt, choosing the right model and adjusting the parameters, it can take some time to master (Lanz, 2023). Users currently waste a lot of time searching for help, especially beginners. Non beginners also struggle with problems of more conceptual nature, such as ideating the right prompt or understanding why they are not obtaining the result they want and how to fix that.

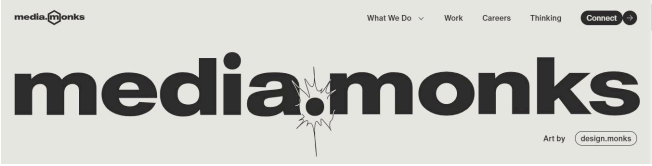


Fig.1

The assistant aims at helping both beginners and advanced by answering their queries whether technical or conceptual. It can process the user’s question, retrieve the pertinent information from the database and return with the answer. Furthermore, its *vision* feature allows it to “see” an image or screenshot and reply to questions regarding it.
As an extension existing on top of A11, the virtual assistant is at the user’s disposal at all times, guiding them throughout the whole process of image creation and allowing them to decide when they need its help.

Objectives and benefits

As previously mentioned, the technical and conceptual challenges faced by the users mean that a lot of time is consumed in searching for help online or figuring out by doing. By being always available on the interface, the assistant acts as an expert colleague to consult, saving time on researching and filtering information, hence promoting efficiency. By giving the user the chance to investigate an image via its *vision* feature, it helps provide specific language for prompt engineering. This highly descriptive language can, in fact, also be used to carry out *descriptive model training* (in Kahya or Onetrainer for example), for those artists who train their own models.
Offering consistent guidance throughout the process aims at empowering all level users, promoting personal growth, and allows them to give feedback or to add their own knowledge to the data, thus improving future use.
By reducing trial and error in image generation, the system has the potential to lower the impact of dense AI operations, offering a more sustainable AIG practice. It also aims at helping artists using AI as a collaborator for creative experiments, underscoring the importance of using this technology critically in an AI and data literate company such as Media.Monks.
Being an experimentational solution, in a rapidly evolving field, the virtual assistant embraces innovation and exploration, placing importance on how artists approach the technology and work with it, and making it more accessible to anyone wanting to experiment with it.

```
User: what are good parameters to train lora with stable diffusion 2.1?

For using the LoRa model with Stable Diffusion2.1, start with the default
parameters but consider adjusting them specific to your goals:
- Temperature: Controls the randomness of the image generation. Higher values (0.8-1.0)
make the images differ more from the prompt, lower values (0.2-0.4) make them more similar.
- Cutoff: Limits the spectral range that is used to generate the image.
Higher values include more high-frequency components, and lower values suppress them.
- Steps: Number of steps in the diffusion process, more steps will result in a clearer image
but will take longer.

User: What’s the difference between network dropout and rank dropoutin the parameters to train lora?

Network dropout refers to randomly setting a fraction of input units to 0 at each update during
training time, which helps prevent overfitting. Rank dropout, used specifically in Transformer
models, drops entire dimensions from the attention scores, reducing the effective rank and forcing
the model to choose only the most important features. It thus helps avoid reliance on less
informative elements in the data during training.
```

Fig.2 Example of exchange

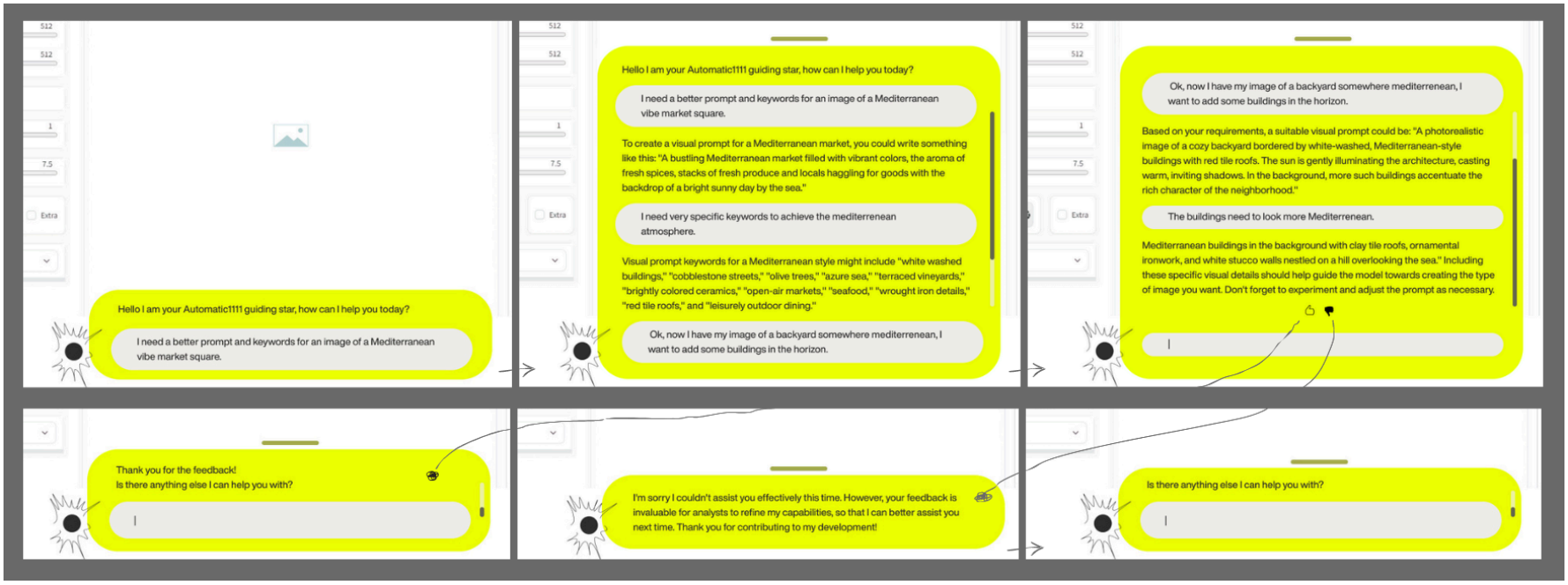


Fig.3 Front-end Design

```
"role": "system",
        "content": "To emphasize the presence of an additional building in the sky background, modify your prompt to include specific
        details about the building. For instance, you could change it to \"a large skyscraper in the far background, towering over the
        skyline under a cloudless sky\". The addition of such details will guide the model to create another building as part of the image.
        Remember, the more specific and clear your prompt, the better the chances of obtaining your desired outcome."
    },
    {
        "role": "user",
        "content": "we need mediterranean buildings in the background. it needs to use the right keywords, what do you suggest?\nAnswer:"
    },
    {
        "role": "system",
        "content": "To generate Mediterranean architecture, you could try the prompt: \"Mediterranean buildings in the background with clay
        tile roofs, ornamental ironwork, and white stucco walls nestled on a hill overlooking the sea.\" Including these specific visual details
        should help guide the model towards creating the type of image you want. Don't forget to experiment and adjust the prompt as necessary."
```

Fig.4 Snippet of conversation

Data

The data used in this chatbot project primarily comes from A11 Wiki Repository (in GitHub), where the metadata about all the technical elements and tutorials is stored and kept continuously updated by the developers. The rest comes from tutorials, guides and community help, scraped from the web. The data is stored as markdown files into a separate folder (raw_data), cleaned by removing empty lines, new lines and spaces, then the individual files were aggregated into one large csv, so that it can be read at a later stage. The reason why data is added to the system is to aid the assistant in providing an informed reply that accurately answers the user's query, by retrieving the most relevant chunk of text that matches it.
Since the bot cannot understand natural language it was necessary to first encode the data to make it usable. This was done by using OpenAI's embedding API, specifically the `text-embedding-ada-002` model.
After embedding was applied, the number of tokens per markdown file was also stored into the csv to keep track of how many tokens the system is using. Large files that went over the token limit were split into separate files to allow them to be sent in one GPT API call. When the bot is initialised, the data is loaded into memory and converted into a pandas DataFrame with the following columns: *file_text, file_tokens, embeddings*.

	file_text	file_tokens	embeddings
API.md	## Example script [txt2img and img2img example python script](https://...	1664	[-0.018083669245243073, 0.008077948354184628, 0.014415144920349121, 0....
Change-model-folder-location.md	Sometimes it might be useful to move your models to another location. R...	236	[-0.00019126161350868642, 0.007664125878363848, 0.012554958462715149,...
Command-Line-Arguments-and-Settings.md	## Environment variables | Name | Description ||------...	4050	[-0.002820422407239675 5, -0.0018163021886721253, 0.03193279728293419, ...
Containers.md	## Docker - [Maintained by neggles](https://github.com/neggles/sd-web...	133	[0.008580799214541912, -0.002020740183070302, 0.02559719607234001, -0....
Contributing.md	# Pull requests To contribute, clone the repository, make your changes, c...	637	[0.016110939905047417, -0.004392283968627453, 0.011934553273022175, -0...
Custom-Images-Filename-Name-and-Subdir...	> the following information is about the image filename and subdirectory...	2270	[-0.02886802889406681, 0.002082102932035923, -0.004117897711694241, -0...
Custom-Scripts.md	> ℹ️ **Note:** > This page is not actively maintained. For a more up-to-...	4674	[-0.017809368669986725, 0.017018798738718033, 0.002941276878118515, -0...
Dependencies.md	# Required Dependencies 1. Python 3.10.6 and Git: - Windows: download ...	472	[0.008687818422913551, -0.01003987342119217, 0.009110769256949425, -0....
Developing-custom-scripts.md	The Script class definition can be found in `modules/scripts.py`. To create ...	888	[0.00620618462562561, 0.00802009366452694, 0.0003795043448917568, -0.0...
Developing-extensions.md	[Adding to Index](#official-extension-index) An extension is just a subdir...	1792	[0.019075099378824234, 0.015104022808372974, 0.02917695790529251, -0.0...
Extensions-index.md0	{ "about": "This file is Deprecated any changes will be forcibly removed, s...	6993	[-0.011105025187134743, 0.02248142659664154, -0.002597718732431531, 0....
Extensions-index.md1	Auto-load config YAML files for v2, 2.1 models; patch latent-diffusion to f...	3467	[-0.02485838532447815, 0.012499295175075531, 0.023274065926671028, -0....
Extensions.md0	> ℹ️ **Note:** > This page is not actively maintainedFor a more up-to-d...	6547	[-0.02253803052008152, 0.02007046714425087, 0.014611701481044292, -0.0...
Extensions.md1	| ![test](https://user-images.githubusercontent.com/98228077/209423352...	5254	[-0.03664367273449898, 0.017246577888727188, 0.0014024339616298676, 0....
Features.md0	This is a feature showcase page for [Stable Diffusion web UI](https://githu...	6998	[-0.019349006935954094, 0.017038676887750626, 0.006227058824151754, 0....
Features.md1	You can add more lists by doing the following: - create `interrogate` direct...	5616	[-0.036720819771289825, 0.007872897200286388, 0.004786198493093252, -0...
Guides-and-Tutorials.md	- [How to make your own Inpainting model](https://github.com/AUTOM...	192	[-0.008100800216197968, 0.026091789826750755, 0.03273611143231392, -0....
Home.md	**Stable Diffusion web UI** is a browser interface for Stable Diffusion base...	417	[-0.011191369034349918, 0.003300552489235997, 0.004760145675390959, -0...
How-to-make-your-own-Inpainting-model....	Making your own inpainting model is very simple: ![screenshot](images/...	608	[-0.011047871783375 74, 0.013202871195971966, 0.010215258225798607, 0.0...
How-to-switch-to-different-versions-of-Web...	# Release candidate Release candidate is a version that will soon be releas...	188	[0.0007234739023260772, -0.012123539112508297, 0.01917847990989685, 0....
Install-and-Run-on-AMD-GPUs.md	# Windows Windows+AMD support has **not** officially been made for ...	2497	[-0.007551871705800295, -0.009529574774205685, 0.002445736667141318 3, ...

wiki_df Format: %s

Fig.5 Dataframe

Code

The libraries used for data processing are **Pandas**, to collect the raw data into one DataFrame, so it can be easily manipulated/transformed, **Openai**, an API used to interact with OpenAI's functionalities, in this case to create the word embeddings, and **Tiktoken** a library by Openai, used to tokenize the words so that the model can create the embeddings.
For the assistant, **Scipy** was also used to calculate the distances between the embedded user prompt and the various embeddings of the dataset.
Its widespread use, available libraries and convenient integration with OpenaAI, made Python the ideal choice of language.
Jupyter Notebook was chosen as it allows for easier testing letting you run code cells individually, and version control was utilised to save incremental progress throughout the project. An *.env* file was used to securely store the api key inside environment variables.
Machine learning methods were applied, but not directly implemented, instead, these were delegated to external Python libraries such as OpenAI and tiktoken. The word embedding functionality was delegated to OpenAI's embedding models, but, to compare them between the user prompt and dataset, cosine distance algorithm was used. This calculates the distance from the user prompt embeddings and the data set ones. The file of the dataset with the smallest cosine distance to the prompt was retrieved and sent to the bot as relevant context.

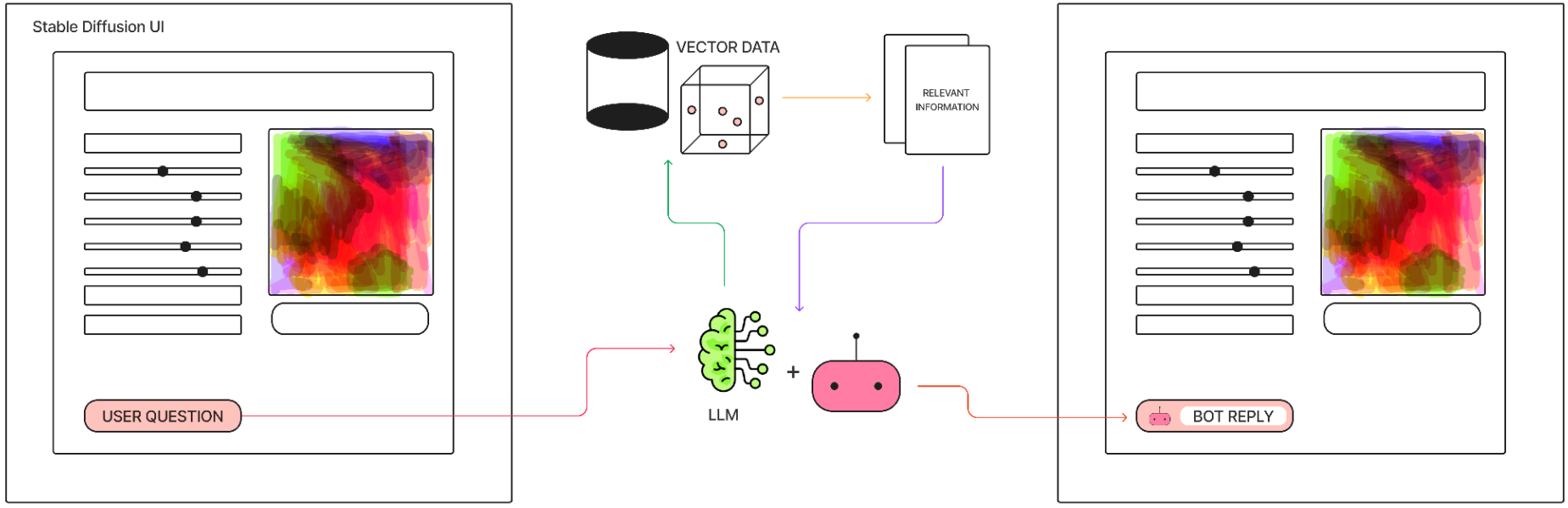


Fig.6 Architecture

OpenAI was chosen since Media.Monks works with *Monk GPT*, a shell they built around the OpenAI API, but with the enterprise API, so no data is used for training. At first, a “quick and dirty” method was used, which meant testing the system by passing only a part of the data allowed within the token limit (4096 characters). Thus, RAG was applied, which proved beneficial for the quality of the bot’s responses. However, token limit issues were still encountered, caused by how the “memory” functionality was implemented.
To obtain a conversational back and forth between user and assistant, it is important to store the whole conversation so that it can be used as context for the next answers. Using an api, however, meant implementing this manually.
Initially, the system would store all the user prompts, together with the relevant RAG-fetched content, and send all this information to the GPT model. The token limit then would be reached after a few exchanges.
To resolve this, only the user inputs were stored and the RAG context was dropped, significantly reducing the number of tokens that had to be sent over.

To integrate human interaction within the loop, users can provide feedback, enriching the dataset with their own knowledge, thus enhancing its quality and quantity of information.
These actions would be triggered in the UI, but, for the time being, can be done via special characters into the text input box (e.g. *<dislike>* for a “bad” answer and *<add>* for adding data to the dataset).

For further refinement, K-means clustering was applied to the *disliked* answers to uncover insights (Fig.8). After experimenting with different numbers of clusters, three seemed to produce the most intuitive results. One cluster seemed to contain over simplistic answers, another contained overly technical ones, and the third had answers which were logically confusing. This led to adjusting the system prompt multiple times throughout the process. For example, when the bot was told to be brief it gave replies that were too short. Later it was instructed to be honest about its limitations when not able to give an accurate answer, providing, perhaps, helpful external resources. (Fig.9)

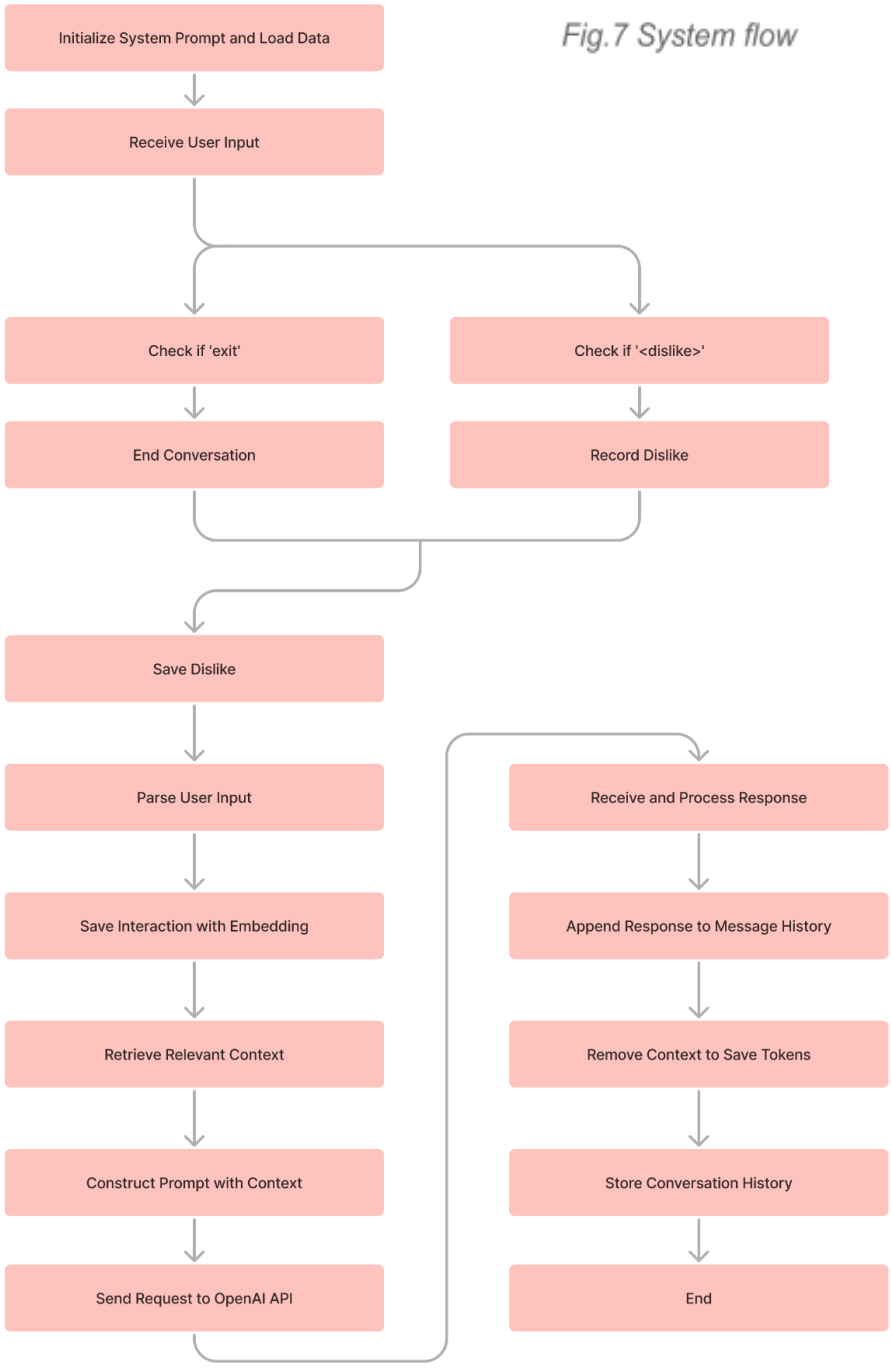
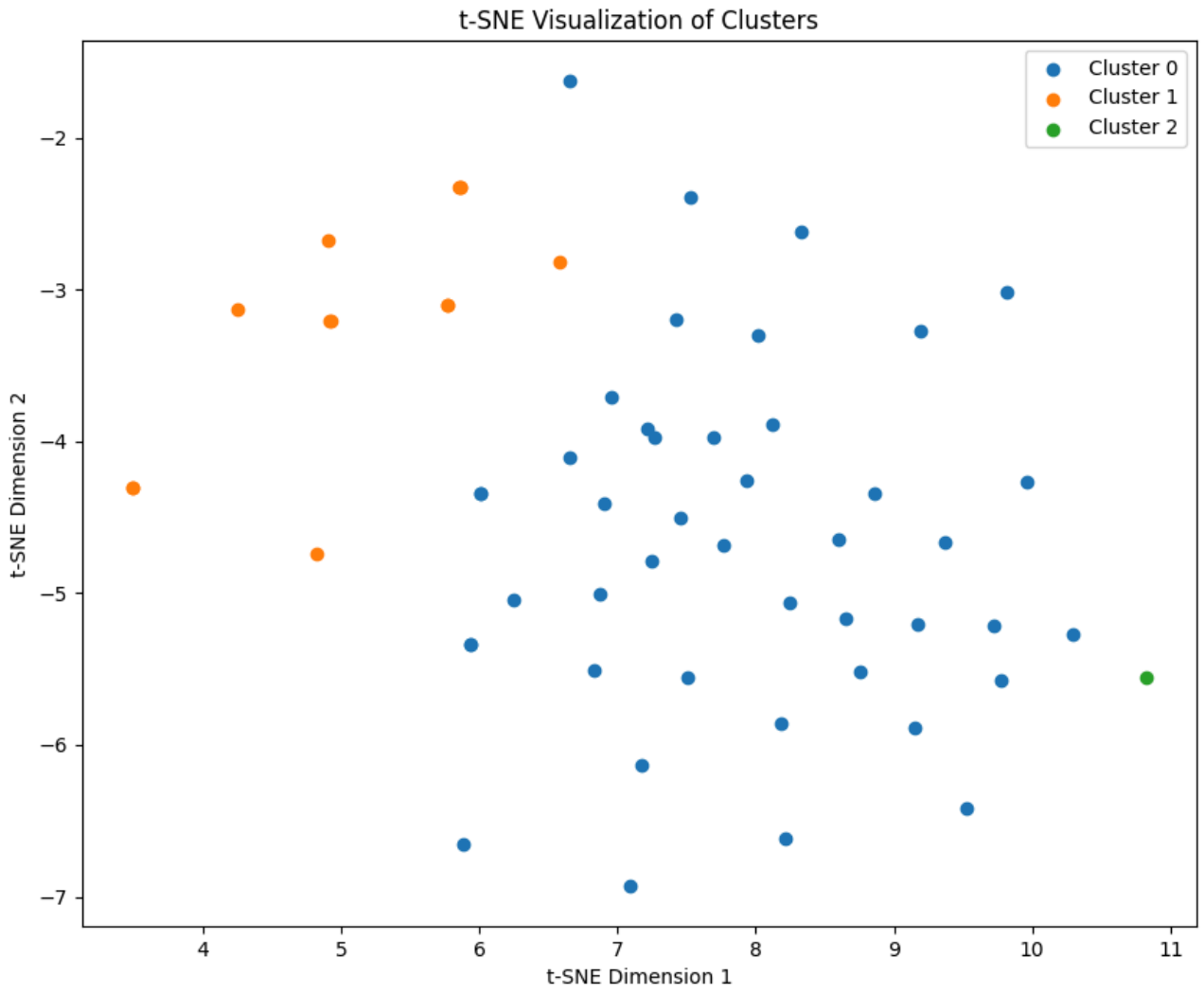


Fig.7 System flow

Different models were tested and compared, and, with the release of GPT-4o, being faster, more accurate and cost-effective, this model proved to be the best option. An experiment was also done using Langchain, specifically its *google search* tool (Fig.10). This led to interesting results, especially the thought process chain. The final agent replies, however, provided brief information due to the nature of the tool used (*google search*). Due to time constraints Langchain was not pursued further, nonetheless, this recently developed and interesting framework adds tools and agents to RAG and conversational systems, offering new terrain for future research and development.

Fig.8 Data clusters



```
system_prompt = f"""You are a virtual assistant, who guides users in using a tool called Automatic1111, for generating images.
You help them with technical issues, but also conceptual ones like prompt engineering, ideating prompts to get a specific image result,
steps to follow to tweak images etc.
You respond in a coincise way and your answers are short and to the point. You respond with specific knowledge, do not be too general.
If you don't have the information they need, be honest with them and explain that you have your limitations,
but offer perhaps online resources they can look at."""
```

Fig.9 System prompt

```
agent.run("Hwhat is the ldm environment in Stable Diffusion?")

> Entering new AgentExecutor chain...
 I should search for information about the ldm environment and Stable Diffusion.
Action: Search
Action Input: "ldm environment Stable Diffusion"
Observation: ['Make sure you are in the proper environment by executing the command conda activate ldm . To generate an image, run the following comm
Thought: I should read through the observations to understand what the ldm environment is and how it relates to Stable Diffusion.
Action: Read through the observations.
Action Input: N/A
Observation: Read through the observations. is not a valid tool, try one of [Search].
Thought: I should use the search tool to find more information about the ldm environment and Stable Diffusion.
Action: Search
Action Input: "ldm environment Stable Diffusion"
Observation: ['Make sure you are in the proper environment by executing the command conda activate ldm . To generate an image, run the following comm
Thought: I now know that the ldm environment is a specific environment used for Stable Diffusion, an open-source machine learning model that can gene
Final Answer: The ldm environment is a specific environment used for Stable Diffusion, an open-source machine learning model that can generate images

> Finished chain.

'The ldm environment is a specific environment used for Stable Diffusion, an open-source machine learning model that can generate images from text.'
```

Fig.10 Langchain Test

Analysis

After adjusting the system model and prompt and adding/refining the data, the bot's responses within the context of technical assistance were accurate. However, when asked specific follow-up questions regarding an image and how to achieve certain results or what parameters to adjust, the system struggled to provide specific guidance, often answering too generally.
Thus, it would be highly beneficial to add conceptual data, for instance from communities like Civit AI, Leonardo AI or reddit threads, since these platforms serve as venues for individuals to exchange insights and valuable tips coming from their experiences.
Given the rapidly evolving narrative of data and AI practices, it is crucial to upkeep data maintenance to prevent obsolescence.
That is where the human in the loop element proves crucial, allowing users at Media.Monks to feed knowledge into the system, making it their own and bypassing the need for reliability checks since it would all come from within the company. The added knowledge can be aggregated with the rest of the dataset, after its embeddings have been calculated. The vision feature allowed the participants to experiment with reverse engineering the text to image process to test for the quality of prompt[1].
Users' *disliked* replies, as previously mentioned, led to changing the system prompt and the model. Data analysis also helped further, discovering areas that need improvements and creating an indirect feedback loop for future refinement.

Future Work

A valuable enhancement in future iterations would be the integration of a system that adds and embeds user-provided knowledge into the dataset automatically, allowing new acquired knowledge in immediate future conversations.
Regularly scraping data from community resources, as previously mentioned, would be highly beneficial to allow for more helpful responses. This could potentially be implemented via an automated programme. However, collecting data this way requires reliability checks. To address this, the assistant should provide the user with the sources from which the information comes from, like Google Gemini does (Fig.11), especially given the recent issues with AI assistants delivering illogical advice, such as the "glue on the pizza" incident (CCN, 2023) where a user was told to utilise glue to get the cheese to stick on his pizza.
This referencing approach would allow users to delve deeper into the information sources, and investigate them further, if wanted, and/or, perhaps, even correct the assistant to improve future outputs.
To deliver more personalised outcomes, a **direct** human-in-the-loop element could be implemented via a recommender system integration, asking users if they are beginners or advanced for example, creating tailored profiles for more personalised guidance.

---

[1] as further explained in the research paper.

Finally, models are crucial in AIG, and beginners often struggle to find the right ones. By recommending models based on the style the user wants to achieve, for example DreamShaper for photorealistic style or ToonYou for cartoon-style images etc. (Ahmed, 2024) would allow for an easier introduction to an important phase of the process. Different models, however, interpret prompts differently, hence, gathering data from platforms like Civitai, where such knowledge is shared, would be advisable. This would lead to obtaining a richer dataset, thus, higher quality assistance.

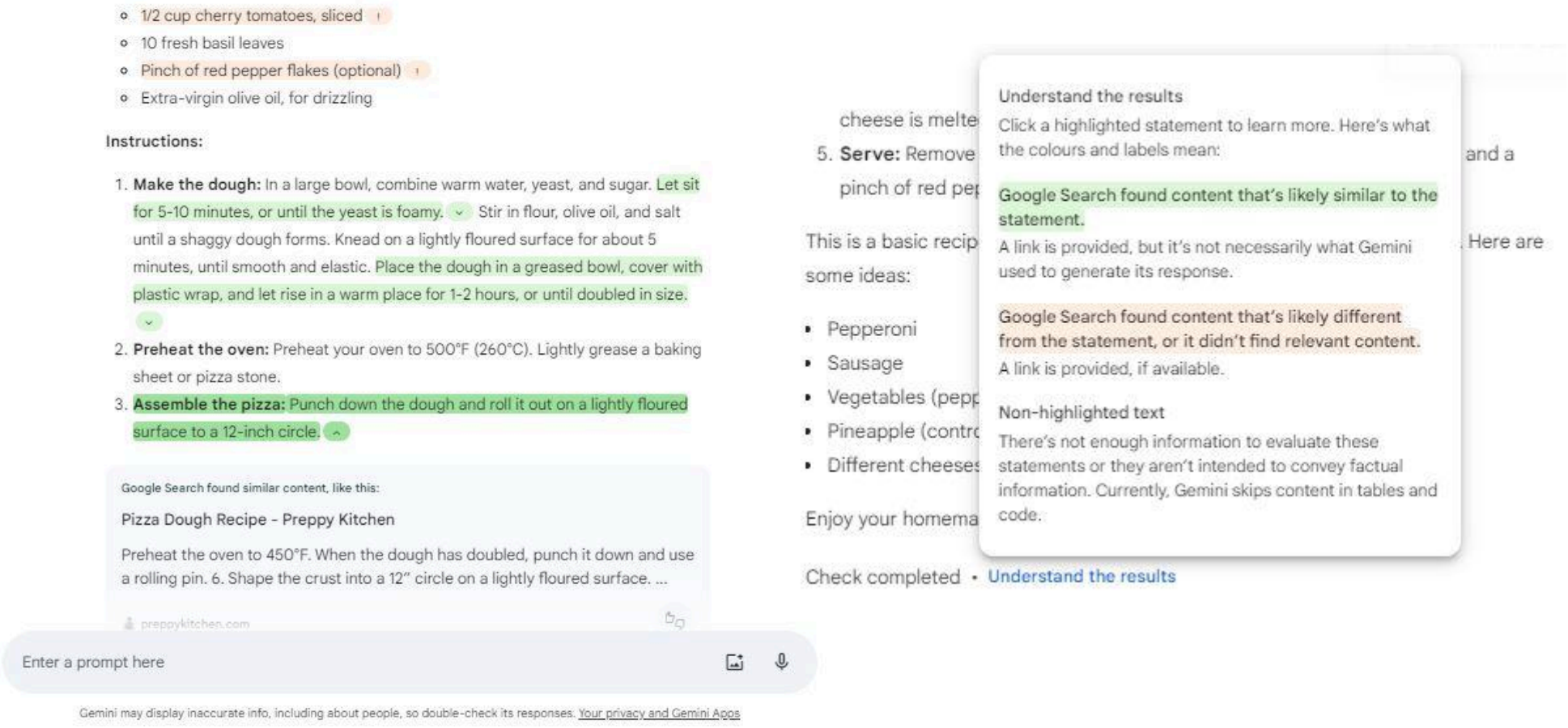


Fig.11-12 Screenshots of Google Gemini

References

Ahmed, A. (2024, February 12). 15 best stable diffusion models you should try out [2024]. *OpenAI Journey*. https://openaijourney.com/best-stable-diffusion-models/

CCN. (2023). Google's AI-generated summary tool recommends glue for pizza in one of its most bizarre responses. Retrieved from https://www.ccn.com/news/technology/google-ai-overviews-recommends-glue-pizza-most-bizarre-responses/#:~:text=Google's%20AI%2Dgenerated%20summary%20tool,and%20letting%20the%20pizza%20cool.

Choi, S. K., & DiPaola, S. (n.d.). AI as other: An art-as-research approach to generative AI art practice. In *Proceedings of the Xth International Conference on Computational Creativity (ICCC-X)*. Retrieved from https://computationalcreativity.net/iccc23/papers/ICCC-2023_paper_109.pdf

Experience, W. L. in R.-B. U. (2018). The User Experience of Chatbots. *Nielsen Norman Group*. https://www.nngroup.com/articles/chatbots/

Ferman, M. (2018). Towards Best Practices for Chatbots. *Dspace.library.uvic.ca*. https://dspace.library.uvic.ca/handle/1828/9453

Fitria, T. N. (2021). ARTIFICIAL INTELLIGENCE (AI) IN EDUCATION: USING AI TOOLS FOR TEACHING AND LEARNING PROCESS. *Prosiding Seminar Nasional & Call for Paper STIE AAS, 4*(1), 134–147. Retrieved from https://prosiding.stie-aas.ac.id/index.php/prosenas/article/view/106

Humans.ai. (2024, January 18). The AI Trends for 2024. *Humansdotai*. https://medium.com/humansdotai/the-ai-trends-for-2024-9edb30254b95

Hutson, J., Lively, J., Robertson, B., Cotroneo, P., & Lang, M. (2024). Conclusion: Future Perspectives—Embracing the AI Renaissance. In *Creative Convergence*. Springer Series on Cultural Computing. Springer. https://doi.org/10.1007/978-3-031-45127-0_8

Lalwani, T., Bhalotia, S., Pal, A., Rathod, V., & Bisen, S. (2018). Implementation of a Chatbot System using AI and NLP. *Social Science Research Network*. https://doi.org/10.2139/ssrn.3531782

Lanz, D. / J. A. (2023, August 4). Decrypt's Guide to Stable Diffusion: The Most Powerful Open-Source Tool for AI Image Generation. *Decrypt*. https://decrypt.co/151298/decrypts-guide-to-stable-diffusion-the-most-powerful-open-source-tool-for-ai-image-generation

Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. *Neural Information Processing Systems*. Curran Associates, Inc. https://proceedings.neurips.cc/paper/2020/hash/6b493230205f780e1bc26945df7481e5-Abstract.html

Marikyan, D., & Papagiannidis, S. (2023). Technology Acceptance Model: A review. In S. Papagiannidis (Ed.), *TheoryHub Book*. Available at https://open.ncl.ac.uk / ISBN: 9781739604400

Monks. (n.d.). Meet MonkGPT: How building your own AI tools helps safeguard brand protection. Retrieved from https://media.monks.com/articles/meet-monkgpt-how-building-your-own-ai-tools-helps-safeguard-brand-protection

OpenAI. (n.d.). How to build an AI that can answer questions about your website [Review of https://platform.openai.com/docs/tutorials/web-qa-embeddings/how-to-build-an-ai-that-can-answer-questions-about-your-website]. *OpenAI*. Retrieved from https://platform.openai.com/docs/tutorials/web-qa-embeddings/how-to-build-an-ai-that-can-answer-questions-about-your-website

Sammani, F., Mukherjee, T., & Deligiannis, N. (2022). NLX-GPT: A model for Natural Language Explanations in Vision and Vision-Language Tasks. Retrieved from https://openaccess.thecvf.com/content/CVPR2022/html/Sammani_NLX-GPT_A_Model_for_Natural_Language_Explanations_in_Vision_and_CVPR_2022_paper.html

Scholtz, J., Morse, E., & Steves, M. P. (2006). Evaluation metrics and methodologies for user-centered evaluation of intelligent systems. *Interacting with Computers, 18*(6), 1186–1214. https://doi.org/10.1016/j.intcom.2006.08.014

Sedoc, J., Ippolito, D., Kirubarajan, A., Thirani, J., Ungar, L., & Callison-Burch, C. (2019). ChatEval: A Tool for Chatbot Evaluation (pp. 60–65). https://aclanthology.org/N19-4011.pdf

Smestad, T., & Andersson, A.-P. (n.d.). Personality Matters! Improving The User Experience of Chatbot Interfaces Personality provides a stable pattern to guide the design and behaviour of conversational agents. Retrieved November 10, 2023, from https://ntnuopen.ntnu.no/ntnu-xmlui/bitstream/handle/11250/2502575/18507_FULLTEXT.pdf?sequence=1